

INSTITUTO TECNOLÓGICO DE SAN JUAN DEL RÍO



TÓPICOS DE CYBERSEGURIDAD

R005 — PARCHADO DE UN EJECUTABLE CON VALIDACIÓN EN LÍNEA

P R E S E N T A:

Angel Elías Romero Paulin

Omar Israel Morales León

Esteban Reséndiz Urquiza

Ingeniería en Sistemas Computacionales

Septiembre de 2026

1. Objetivo

Analizar un ejecutable que realiza una validación de credenciales mediante una comunicación en línea y modificar su flujo de control para que permita el acceso independientemente del usuario y la contraseña introducidos. Además, automatizar el parche mediante un programa en Python que genere una copia modificada a partir del ejecutable original.

2. Alcance y consideraciones éticas

La práctica se realizó exclusivamente sobre el ejecutable de laboratorio proporcionado para la asignatura. El propósito es académico: comprender la relación entre instrucciones de salto, representación binaria y automatización de parches. No se realizaron pruebas contra sistemas ajenos ni se intentó acceder a servicios de terceros fuera del escenario de práctica.

3. Herramientas y archivos utilizados

- x32dbg, para analizar el ejecutable de 32 bits y modificar instrucciones en ensamblador.
- Python 3, para automatizar el parche binario.
- Visual Studio Code, para editar el archivo hackame1.py.
- Símbolo del sistema de Windows, para ejecutar y verificar los archivos generados.
- phackeame.exe, ejecutable original de la práctica.
- phackeame_parcheado.exe, copia obtenida mediante modificación manual.
- phackeame_python.exe, copia generada automáticamente mediante Python.

4. Descripción general del escenario

El ejecutable presenta una interfaz de inicio de sesión con campos de usuario y contraseña. En el binario se identificaron cadenas relacionadas con la validación de acceso, entre ellas referencias a PASS, controles de contraseña y elementos asociados con una petición HTTP de inicio de sesión. El objetivo del análisis fue localizar la decisión que determina si el programa continúa con el acceso o rechaza las credenciales.

4.1 Verificación de los archivos de trabajo

Archivo	Tamaño	SHA-256	Observación
phackeame.exe	2,128,915 bytes	160c6af0d3e86ec0f1eef57f00c84ae6b5969a8a46c7caf1506d12516047a00c	Original
phackeame_parcheado.exe	2,128,915 bytes	d4d04211352c121f9ecd9f683000487d7f819a11e6eab4d25d61fc3aba6ace49	Parche manual
phackeame_python.exe	2,128,915 bytes	431e2bc785b73ddc5c1511f9e91276278a39549793c5400ed5061806cd6baec5	Parche automático

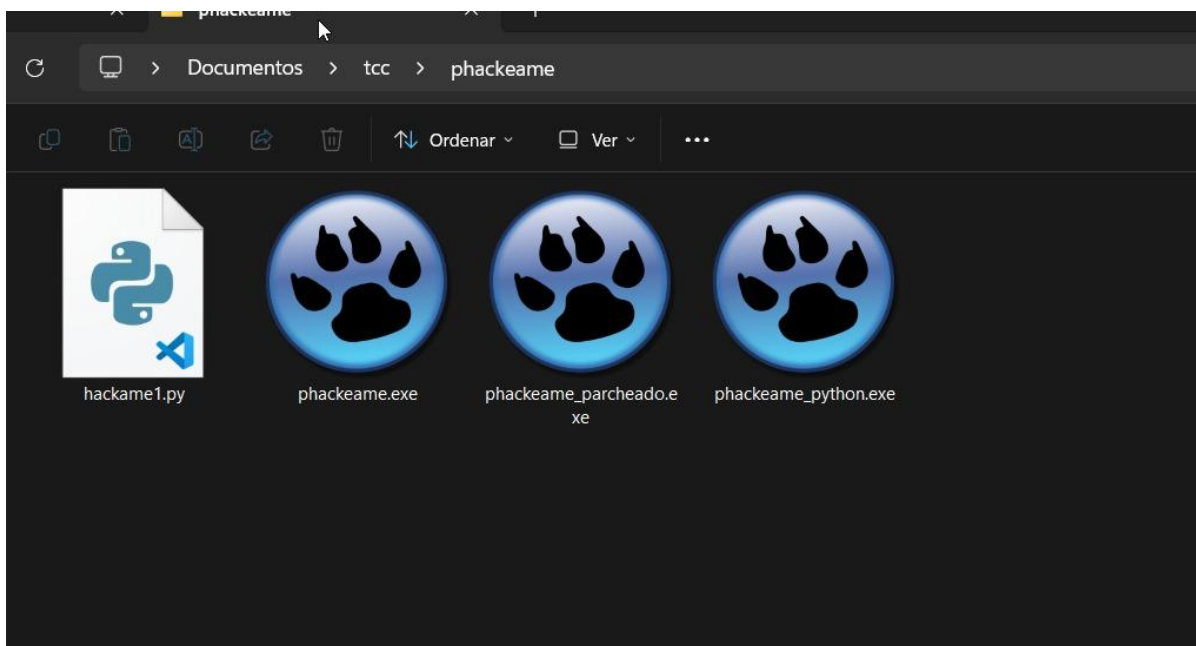


Figura 1. Archivos utilizados durante la práctica: ejecutable original, versión parcheada manualmente, versión generada con Python y script de automatización.

5. Análisis dinámico con x32dbg

El ejecutable original se abrió en x32dbg. Durante el análisis se localizó la instrucción de salto condicional ubicada en la dirección virtual 0x0042A266. La instrucción original corresponde a los bytes 75 1C, interpretados por el depurador como un salto JNE hacia 0x0042A284. Esta decisión forma parte del flujo que separa el camino de validación correcta del camino de rechazo.

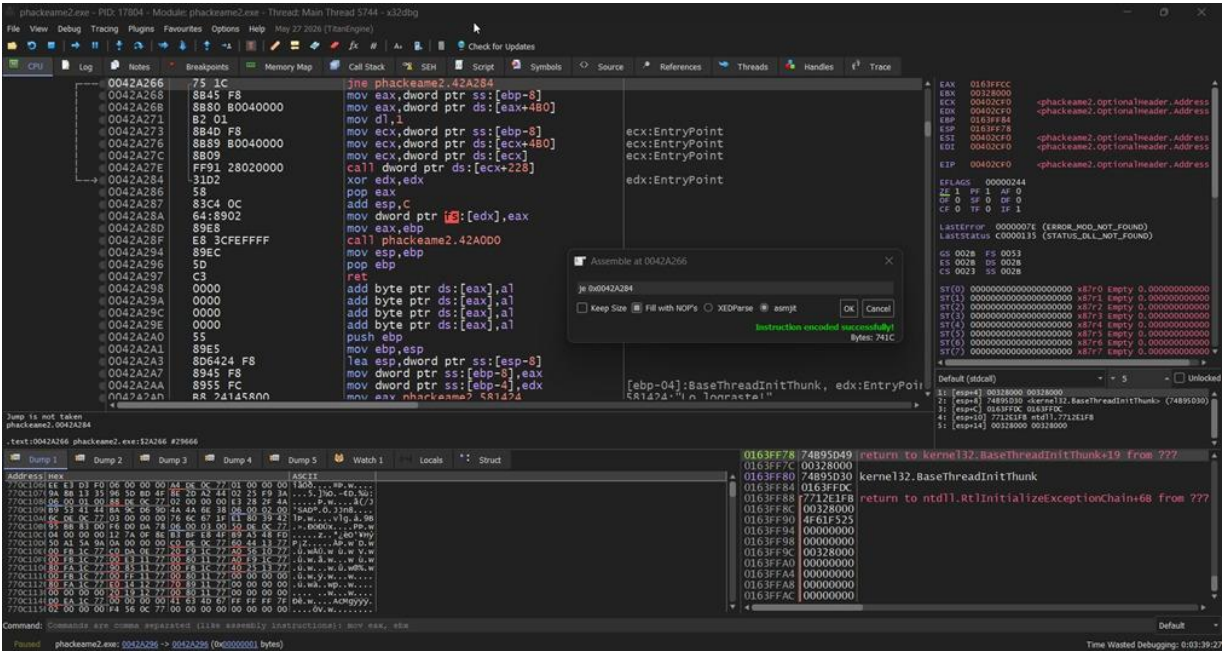


Figura 2. Localización del salto condicional en x32dbg. Se observa la instrucción JNE en 0x0042A266 y la ventana de ensamblado utilizada para modificarla.

5.1 Parche manual

En la modificación manual se sustituyó el primer byte del salto, cambiando 75 1C por 74 1C. El byte 0x75 representa JNE y 0x74 representa JE; por tanto, el desplazamiento del salto se conserva y se invierte la condición. El ejecutable resultante se guardó como phackeame_parcheado.exe.

Referencia	Original	Parche manual	Efecto
Offset de archivo 0x29666	75	74	JNE → JE
Bytes completos	75 1C	74 1C	Se conserva el desplazamiento 1C

Nota: x32dbg muestra una dirección virtual (0x0042A266), mientras que el script de Python trabaja con un desplazamiento dentro del archivo (0x29666). Son referencias distintas al mismo punto lógico del ejecutable y no deben confundirse.

6. Automatización del parche con Python

Después de identificar la ubicación y los bytes relevantes se desarrolló un script que abre el ejecutable original, comprueba que la firma esperada exista en el offset 0x29666 y, únicamente si coincide, sustituye 75 1C por 90 90. Los bytes 90 90 corresponden a dos instrucciones NOP, por lo que el salto condicional queda neutralizado. La salida se guarda como phackeame_python.exe.

```
ORIGEN = Path("phackeame.exe")
DESTINO = Path("phackeame_python.exe")

OFFSET = 0x29666
BYTES_BUSCADOS = b"\x75\x1c"
BYTES_NUEVOS = b"\x90\x90"

contenido = bytearray(ORIGEN.read_bytes())
bytes_actuales = contenido[OFFSET : OFFSET + len(BYTES_BUSCADOS)]

if bytes_actuales == BYTES_BUSCADOS:
    contenido[OFFSET : OFFSET + len(BYTES_BUSCADOS)] = BYTES_NUEVOS
    DESTINO.write_bytes(contenido)
```

Fragmento 1. Lógica principal del parche automático en Python.

6.1 Diferencia binaria producida por el script

Offset	Original	Python	Interpretación
0x29666	75	90	Se elimina el opcode JNE
0x29667	1C	90	Se elimina el desplazamiento del salto

La verificación previa de los bytes evita aplicar el parche sobre un archivo cuya estructura no coincida con la versión esperada. Aun así, el script depende de un offset fijo, por lo que una recompilación o una versión diferente del ejecutable podría requerir localizar nuevamente la instrucción.

7. Pruebas funcionales

Una vez generadas las versiones modificadas se ejecutó el programa y se introdujeron credenciales de prueba. La evidencia muestra que el ejecutable parcheado presenta un mensaje de acceso exitoso aun cuando las credenciales introducidas no corresponden a una validación normal. Con ello se comprobó que la modificación del flujo de control afecta directamente la decisión de acceso del cliente.

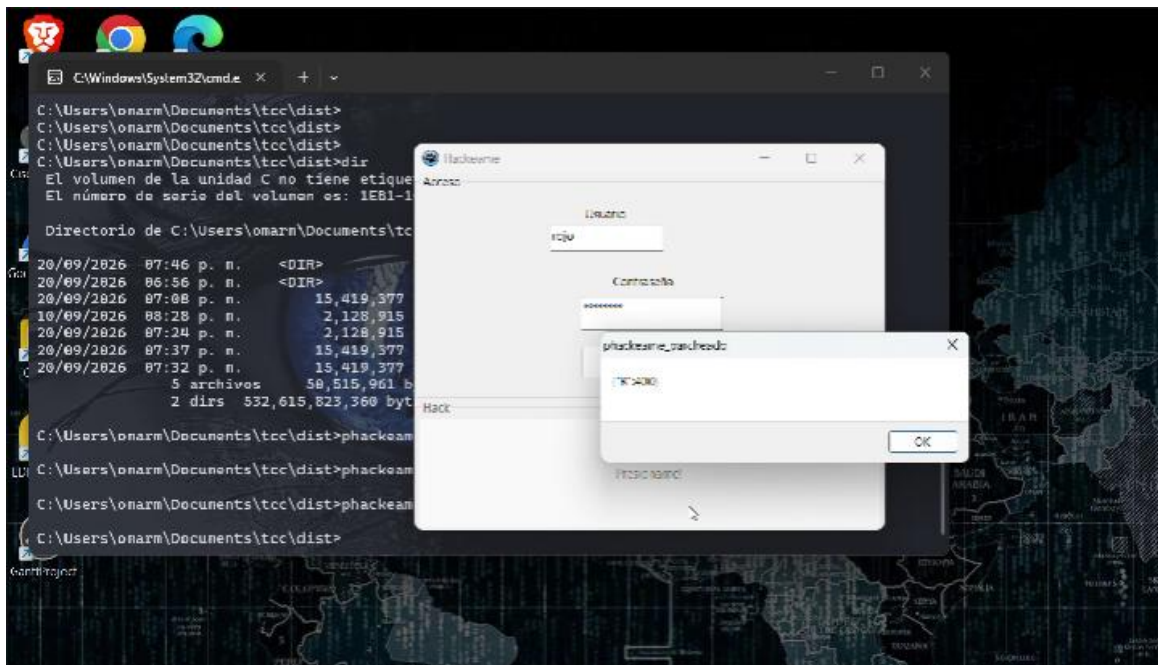


Figura 3. Prueba del ejecutable parcheado con credenciales de ejemplo y mensaje de acceso.

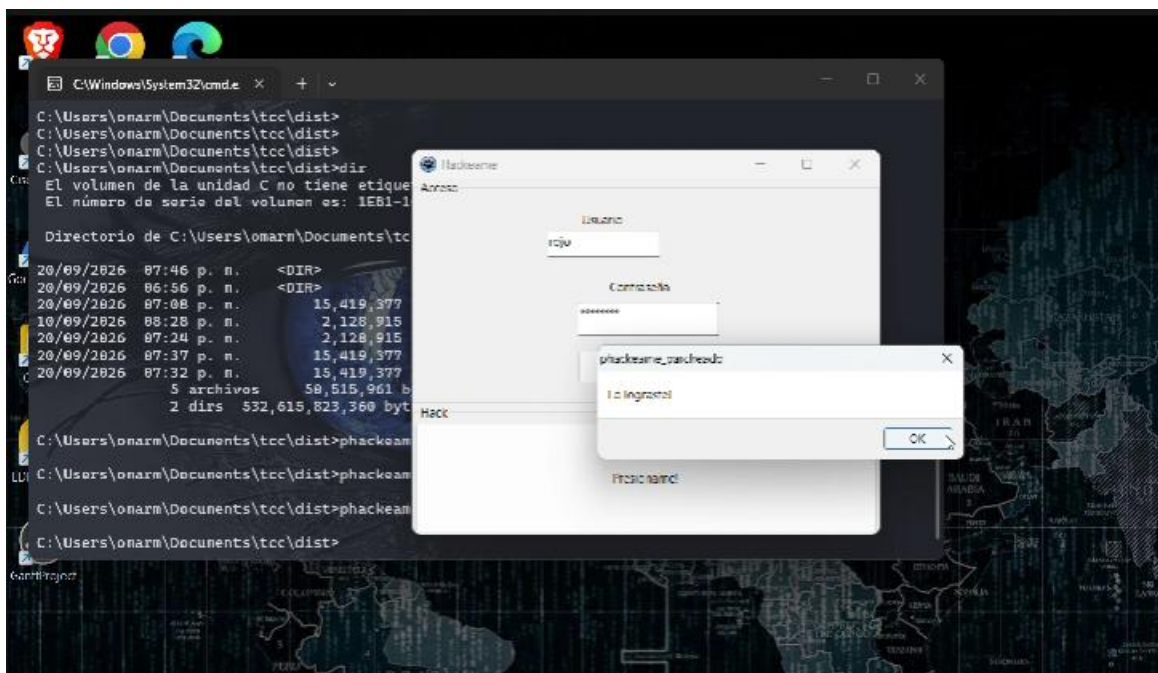


Figura 4. Confirmación visual del acceso después de aplicar el parche.

7.1 Comparación de resultados

Versión	Cambio	Bytes modificados	Resultado
Original	Sin modificación	—	Mantiene la lógica de validación original
Parche manual	JNE → JE	1 byte	Invierte la condición de decisión
Parche Python	JNE → NOP NOP	2 bytes	Neutraliza el salto condicional

8. Resultados

- Se identificó un salto condicional relevante en 0x0042A266 dentro del ejecutable analizado.
- La modificación manual produjo una copia que difiere del original en un solo byte: 75 → 74 en el offset 0x29666.
- El parche automático verificó la secuencia 75 1C y la sustituyó por 90 90.
- Los tres ejecutables conservan el mismo tamaño de 2,128,915 bytes, pero presentan hashes SHA-256 distintos.
- Las pruebas funcionales confirmaron que las versiones parcheadas alteran la decisión de acceso del programa.

9. Conclusiones

La práctica permitió relacionar una instrucción de ensamblador con su representación en bytes y con el comportamiento observable de una aplicación. El análisis con x32dbg mostró que una decisión de acceso puede depender de un único salto condicional. Al invertir dicho salto o reemplazarlo por NOP, el flujo de ejecución cambia sin necesidad de modificar la interfaz o recompilar el programa fuente.

La automatización en Python permitió reproducir el cambio de forma controlada al comprobar previamente los bytes esperados. También se observó que un parche basado exclusivamente en offsets es sensible a cambios de versión, por lo que en un escenario real sería recomendable complementar la verificación con firmas adicionales o hashes de la versión objetivo.

De acuerdo con las indicaciones generales de la actividad, los ejecutables se utilizaron únicamente como material de trabajo y no deben formar parte de la entrega final. El entregable debe conservar el código fuente del parcheador y la documentación del procedimiento.

Anexo A. Código completo de hackame1.py

```
from pathlib import Path

# Configuración adaptada a los archivos de tu carpeta
ORIGEN = Path("phackeame.exe")
DESTINO = Path("phackeame_python.exe")

OFFSET = 0x29666
BYTES_BUSCADOS = b"\x75\x1c"
```

```

BYTES_NUEVOS = b"\x90\x90"

def parchear_ejecutable():
    if not ORIGEN.exists():
        print(f"[-] Archivo no encontrado: {ORIGEN.name}")
        return

    # Leer todo el binario en memoria
    contenido = bytearray(ORIGEN.read_bytes())

    # Extraer los bytes en la posición objetivo
    bytes_actuales = contenido[OFFSET : OFFSET + len(BYTES_BUSCADOS)]
    print(f"[*] Bytes detectados en 0x{OFFSET:X}: {bytes_actuales.hex().upper()}")

    # Verificar coincidencia antes de modificar
    if bytes_actuales == BYTES_BUSCADOS:
        # Reemplazo directo en el buffer
        contenido[OFFSET : OFFSET + len(BYTES_BUSCADOS)] = BYTES_NUEVOS

        # Guardar en el nuevo archivo
        DESTINO.write_bytes(contenido)

        print("[+] Parche aplicado con éxito.")
        print(f"[+] Ejecutable generado: {DESTINO.name}")
    else:
        print("[-] Los bytes encontrados no coinciden con la firma esperada.")

if __name__ == "__main__":
    parchear_ejecutable()

```

Anexo B. Resumen de integridad de archivos

Archivo	Tamaño	SHA-256
phackeame.exe	2,128,915	160c6af0d3e86ec0f1eef57f00c84ae6b5969a8a46c7caf1506d12516047a00c
phackeame_parcheado.exe	2,128,915	d4d04211352c121f9ecd9f683000487d7f819a11e6eab4d25d61fc3aba6ace49
phackeame_python.exe	2,128,915	431e2bc785b73ddc5c1511f9e91276278a39549793c5400ed5061806cd6baec5